

Apuntes
de Clases

2026

Introducción a Scratch



Diego Maldonado

Presentación

Este documento fue elaborado para el Taller de Scratch 2026, ofrecido por las escuelas de Ingeniería en Matemática y Pedagogía en Matemática de la Universidad Católica del Maule. Está dirigido a docentes de enseñanza media, especialmente a profesores de Matemática y Computación.

A lo largo de sus secciones se presentan los aspectos fundamentales para comenzar a utilizar Scratch: desde el registro y la exploración de su interfaz gráfica, pasando por la creación de animaciones mediante bloques de programación y control, hasta la interacción del usuario con los personajes del escenario y el desarrollo de videojuegos sencillos.

Índice general

Capítulo 1

Primeros pasos en Scratch _____ Página 7

- 1.1 ¿Qué es Scratch? 7
- 1.2 Registrarse e iniciar Scratch 7
- 1.3 La interfaz de Scratch 8
- 1.4 El escenario y los personajes 9
 - Explorando los escenarios — 9 • La pestaña fondos — 11 • Edición de personajes — 12
- 1.5 Posicionamiento de objetos y escenarios 13
 - Personajes y panel de objetos — 14 • Propiedades de un personaje — 15 • Orientación de un personaje — 15 • Agregar, seleccionar, duplicar y eliminar personajes — 16 • Transformaciones y orden visual — 16

Capítulo 2

Animaciones _____ Página 17

- 2.1 Mis primeros bloques 17
 - El bloque Evento al presionar  — 19 • Bloques de Movimiento — 19 • Bloques de Apariencia — 20 • Bloques de control I — 21 • Bloques de sonido* — 23
- 2.2 El escenario: bloques y fondos 24
 - Los fondos — 25 • Bloques de apariencia del escenario — 25 • Fondos y bloques de control — 27 • Eventos asociados a los fondos — 28 • Inicialización del escenario — 29

Capítulo 3

Videojuegos _____ Página 31

- 3.1 Interacción con el jugador 31
 - Bloques de control II y booleanos — 31 • Eventos de teclado — 35 • Uso del ratón — 36 • Movimiento controlado — 37
- 3.2 Sensores y colisiones 39
 - Bloques de sensores — 39 • Operadores — 39 • Colisiones — 40
- 3.3 Variables y reglas de juego 41
 - Introducción a las variables — 41 • Puntaje — 41 • Vidas y energía — 42 • Temporizadores — 43
- 3.4 Azar y movimiento automático 44
 - Números aleatorios — 44 • Enemigos y obstáculos — 45 • Dificultad progresiva — 45
- 3.5 Comunicación entre personajes 46
 - Mensajes — 46 • Organización del juego mediante mensajes — 47 • Secuencias coordinadas — 47

3.6	Estados y pantallas del videojuego	48
	Pantalla de inicio — 48 • Estado de juego — 49 • Pantalla de victoria — 49 • Pantalla de derrota — 49 • Cambio de niveles — 50	
3.7	Repeticiones avanzadas y control del juego	51
	El bloque repetir hasta que — 51 • Detención de programas — 51 • Esperar condiciones — 52 • Ciclos de juego — 52	
3.8	Clones	52
	Introducción a los clones — 53 • Aplicaciones en videojuegos — 53 • Propiedades de cada clon — 54	

Capítulo 1

Primeros pasos en Scratch

1.1. ¿Qué es Scratch?

Es un lenguaje de programación creado por el Massachusetts Institute of Technology (M.I.T.) para desarrollar juegos, creaciones interactivas para ocio y educación.

A diferencia de los lenguajes típicos de programación, con Scratch los programas se crean mediante bloques constructivos que se encajan de la misma manera que el montaje de un juego de Lego. Otra ventaja de este enfoque es que los resultados se pueden visualizar de inmediato lo que facilita la corrección de los errores y el propio aprendizaje.

1.2. Registrarse e iniciar Scratch

Para crear una cuenta en Scratch, sigue las instrucciones del diagrama de la figura 1.1.

Registrarse te permitirá:

- Guardar tus proyectos en línea.
- Acceder desde cualquier computador con conexión a Internet.
- Compartir tus creaciones con la comunidad.
- Explorar y reutilizar proyectos de otras personas.

Nota: Necesitas tener un correo electrónico.

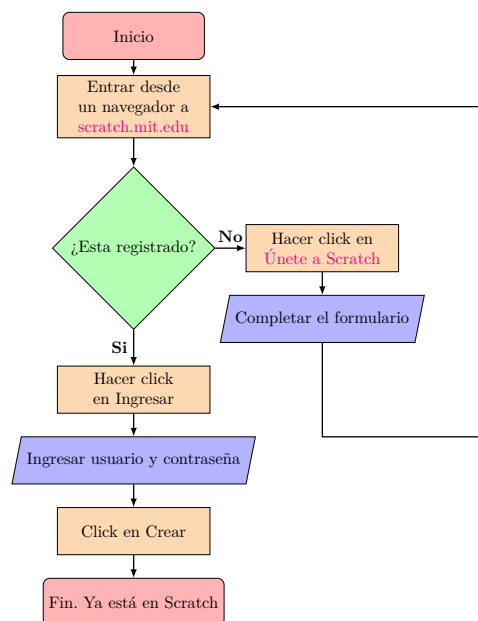


Figura 1.1: Diagrama para registro e inicio de sesión de Scratch.

1.3. La interfaz de Scratch

Una vez haya iniciado sesión y hecho click en *crear* vera la siguiente pantalla:

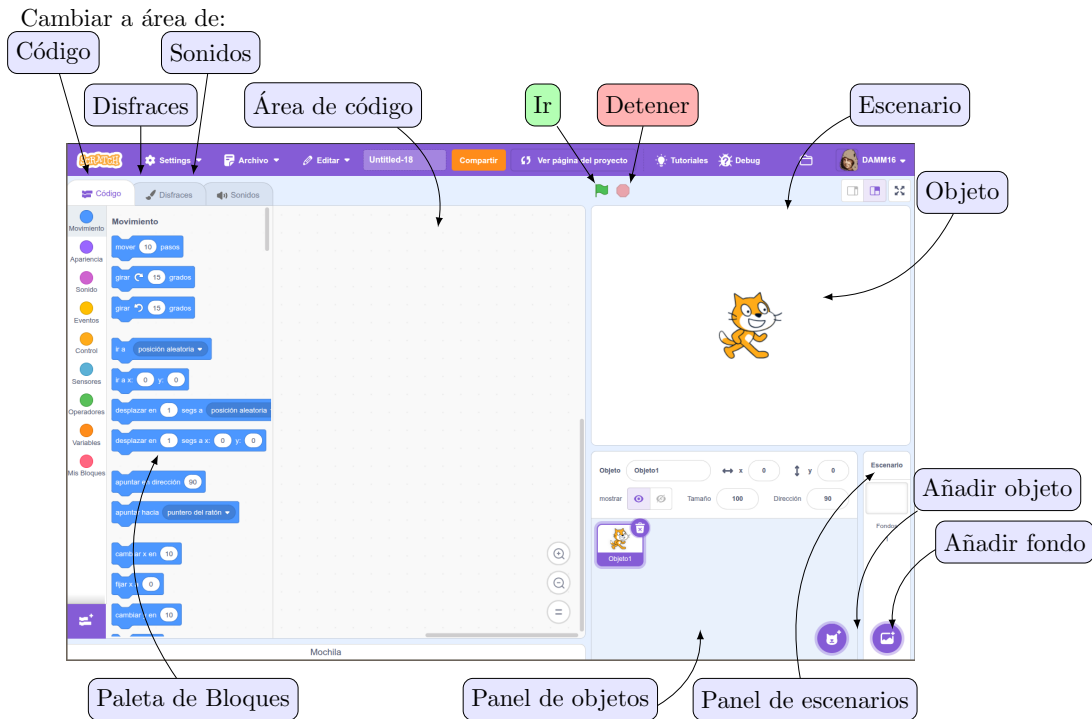


Figura 1.2: Interfaz de Scratch

En esta interfaz, los elementos principales son:

- **Escenario:** Zona principal donde se muestran las animaciones y juegos. Es el “teatro” donde ocurre la acción del programa. Aquí se visualizan los personajes, fondos y movimientos.
- **Objeto, personaje o sprite:** Elemento individual que puede ser controlado mediante programación. Cada objeto puede tener disfraces, sonidos y su propio conjunto de instrucciones.
- **Área de código:** Espacio donde se construyen los programas arrastrando y conectando bloques. Cada objeto tiene su propia área de código.
- **Código:** Conjunto de bloques organizados que definen el comportamiento de un objeto. Indican qué hacer, cuándo hacerlo y cómo responder a eventos.
- **Disfraces:** Apariencias que puede tener un objeto. Permiten cambiar su forma, color o posición para crear animaciones dinámicas.
- **Sonidos:** Archivos de audio que pueden asociarse a los objetos para enriquecer las animaciones y juegos. Se pueden grabar, importar o editar.
- **Paleta de Bloques (de código):** Elementos visuales que representan instrucciones de programación. Están agrupados en categorías como Movimiento, Apariencia, Sonido, Eventos, Control, Sensores, Operadores y Variables.

- **Panel de objetos:** Sección que muestra todos los objetos presentes en el proyecto. Desde aquí se pueden seleccionar, renombrar, duplicar o eliminar.
- **Panel de escenarios:** Zona donde se administra el fondo del escenario. Permite cambiar y gestionar diferentes fondos para ambientar la escena.
- **Añadir objeto:** Botón que permite incorporar nuevos personajes u objetos al proyecto. Se pueden elegir de la biblioteca, dibujar, cargar desde archivos o usar la cámara.
- **Añadir fondo:** Botón que permite cambiar o agregar fondos al escenario. Los fondos definen el entorno visual donde ocurre la acción.
- **Botón Ir:** Boton que al presionarse inicia el juego o animación.
- **Botón Detenr:** Boton que al presionarse detiene el juego o animación.

1.4. El escenario y los personajes

El escenario es el área donde se desarrolla la acción de un proyecto en Scratch. Es el “teatro” donde los personajes (también llamados “sprites”) interactúan, se mueven y realizan diversas acciones. El escenario puede tener diferentes **fondos** que cambian la apariencia del entorno, y los personajes pueden tener múltiples **disfraces** que alteran su apariencia. Además, el escenario puede incluir efectos visuales y sonoros que enriquecen la experiencia del usuario.

1.4.1. Explorando los escenarios

Al dejar el cursor del ratón sobre el botón de *fondos* () se desplegará un menú con las siguientes opciones:

Subir un fondo: Permite cargar un fondo desde un archivo en tu computadora.

Sorpresa: Selecciona un fondo aleatorio de la biblioteca de Scratch.

Pintar: Abre el editor de fondos para crear un fondo personalizado.

Elige un fondo: Permite seleccionar un fondo de la biblioteca de Scratch.

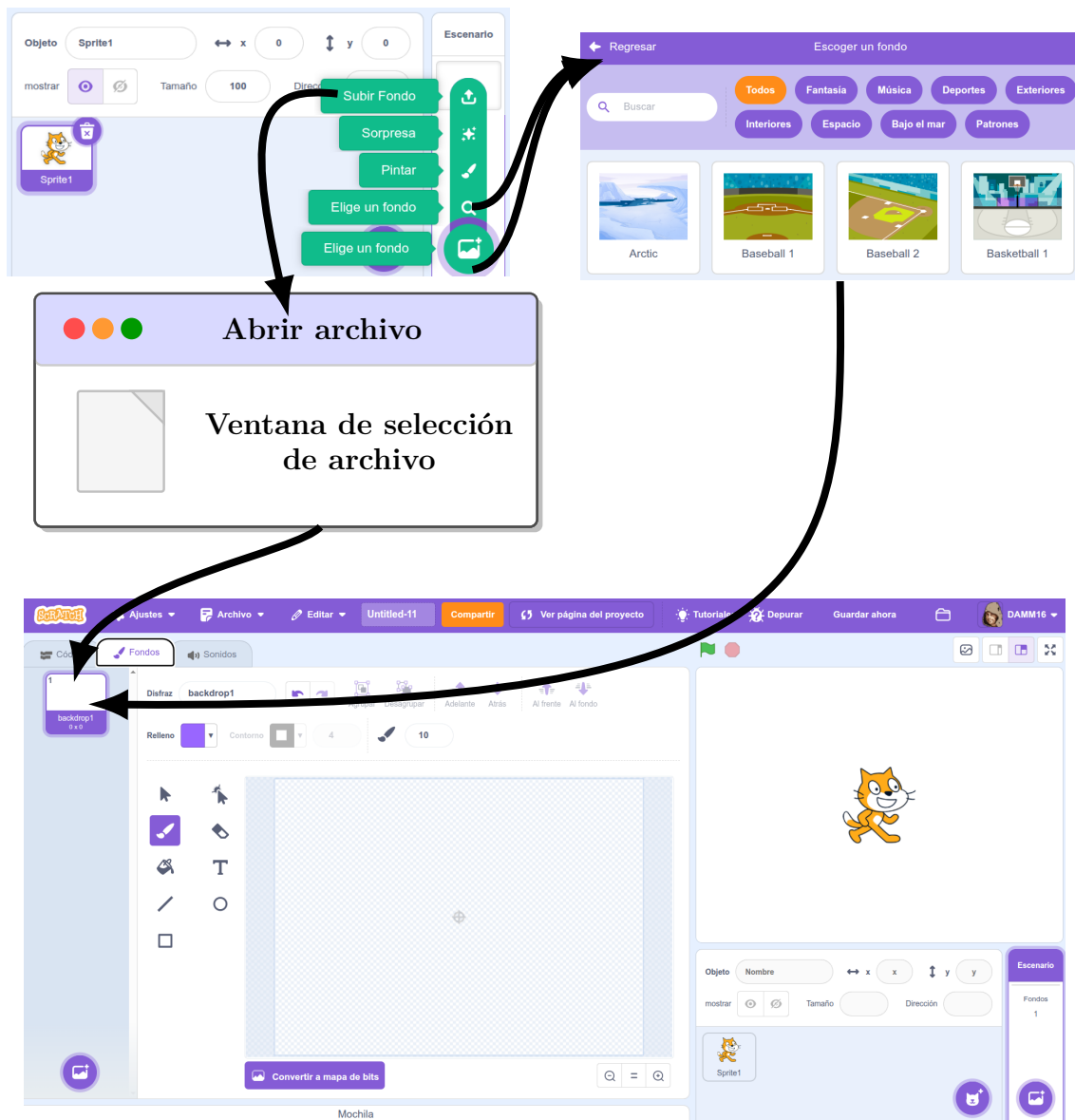


Figura 1.3: Flujo de acciones para cargar un fondo en Scratch.

Ejercicio 1.1

1. Explora las opciones de la pestaña de fondos y prueba a cambiar el fondo del escenario.
2. Crea un fondo personalizado usando la opción de pintar y dibuja un paisaje simple.
3. Carga un fondo desde tu computadora y observa cómo se integra en el escenario.

1.4.2. La pestaña fondos

Al hacer click en el escenario del **panel de escenarios** la pestaña **Disfraces** pasa a ser **Fondos** y se despliega el **editor de fondos**. En este editor se pueden crear, modificar y gestionar los fondos del escenario. Permite dibujar, importar imágenes, cambiar colores, añadir texto y aplicar efectos visuales para personalizar el entorno del proyecto.

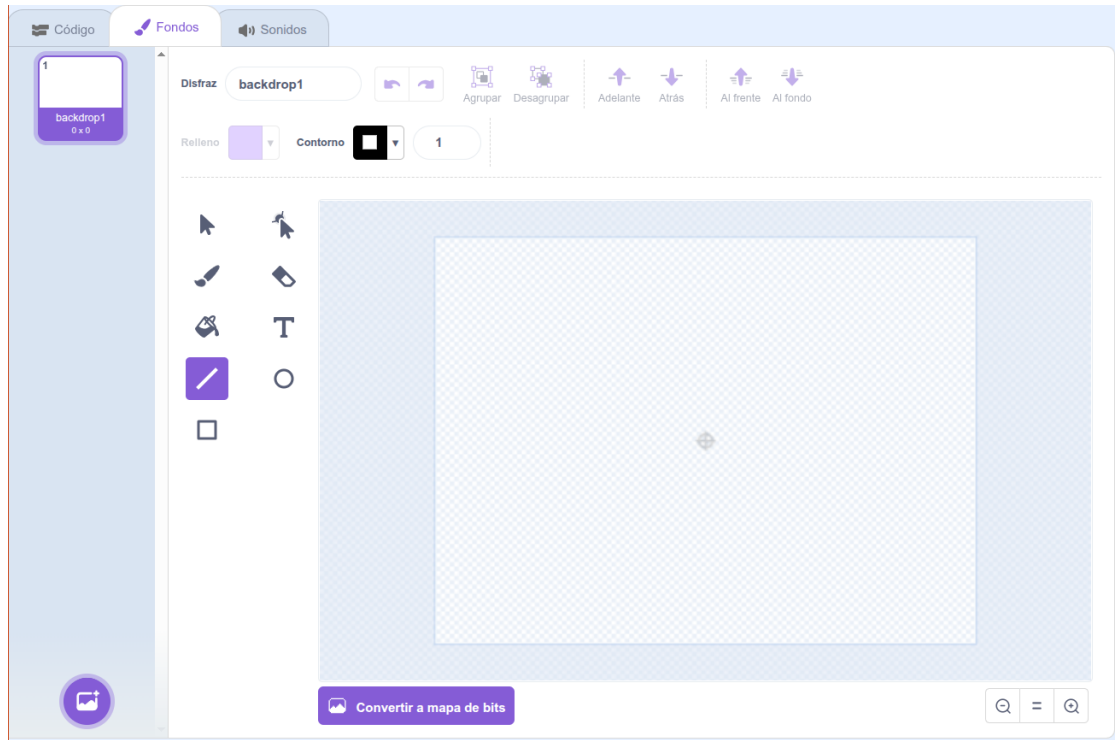


Figura 1.4: Interfaz de Scratch

El editor de fondos tiene dos modos de uso, el modo **mapa de bits** y el modo **vectorial**. El modo mapa de bits permite trabajar con imágenes rasterizadas, mientras que el modo vectorial permite crear gráficos escalables y editables. Los usuarios pueden cambiar entre estos modos según sus necesidades de diseño y edición.

La principal diferencia entre estos modos radica en cómo se representan los gráficos y cómo se pueden manipular. En el modo mapa de bits, los gráficos se componen de píxeles individuales, lo que permite un control detallado sobre la apariencia visual, dibujando pixel a pixel una imagen. En cambio, en el modo vectorial, los gráficos se definen mediante fórmulas matemáticas, lo que permite escalarlos sin pérdida de calidad y facilita la edición de formas y líneas.

Modo mapa de bits:

En este modo, los fondos se representan como una cuadrícula de píxeles. Es ideal para trabajar con imágenes detalladas y efectos visuales complejos. Las herramientas disponibles son:

- Pincel: Permite dibujar a mano alzada sobre el fondo.
- Línea: Permite dibujar líneas rectas.
- Círculo: Permite dibujar círculos y elipses.

- Rectángulo: Permite dibujar rectángulos y cuadrados.
- Texto: Permite añadir texto al fondo.
- Relleno: Permite rellenar áreas con un color sólido.
- Borrador: Permite eliminar partes del fondo.
- Selección: Permite seleccionar y mover partes del fondo.

Modo vectorial:

En este modo, los fondos se representan mediante formas geométricas y líneas. Es ideal para crear gráficos escalables y editables. Las herramientas disponibles son:

- Selección: Permite seleccionar y mover objetos vectoriales.
- Cambiar forma: Permite modificar la forma de los objetos vectoriales.
- Pincel: Permite dibujar a mano alzada con trazos vectoriales.
- Borrador: Permite eliminar objetos vectoriales.
- Relleno: Permite rellenar objetos vectoriales con colores sólidos o degradados.
- Texto: Permite añadir texto como objeto vectorial.
- Línea: Permite dibujar líneas rectas como objetos vectoriales.
- Círculo: Permite dibujar círculos y elipses como objetos vectoriales.
- Rectángulo: Permite dibujar rectángulos y cuadrados como objetos vectoriales.

Ejercicio 1.2

1. Crea un fondo simple, un paisaje por ejemplo, usando el modo vectorial.
2. Crea un segundo fondo, usando el modo mapa de bits.

Explora las herramientas de edición en ambos modos y describe las diferencias que encuentras al trabajar con cada uno.

1.4.3. Cargando y editando personajes

Al seleccionar un personaje en el **panel de objetos**, la pestaña  **Disfraces** se despliega mostrando el **editor de disfraces**. En este editor se pueden crear, modificar y gestionar los disfraces del personaje. Los **disfraces** son distintas apariencias de un personaje. Un personaje puede tener múltiples disfraces, lo que facilita la creación de animaciones y efectos visuales.

Al dejar el cursor del ratón sobre el botón de *objetos* () se desplegará un menú con las siguientes opciones:

Subir un objeto: Permite cargar un personaje desde un archivo en tu computadora.

Sorpresa: Selecciona un personaje aleatorio de la biblioteca de Scratch.

Pintar: Abre el editor de disfraces para crear un personaje personalizado.

Elige un objeto : Permite seleccionar un personaje de la biblioteca de Scratch.

Estos llevan la pestaña  **Disfraces** a mostrar el **editor de disfraces**, donde se pueden crear, modificar y gestionar los disfraces del personaje. Los disfraces permiten cambiar la apariencia del personaje, facilitando la creación de animaciones y efectos visuales de la misma manera que se crean y editan los escenarios.

Ejercicio 1.3 (Animación de un personaje)

La animación de un personaje se puede lograr cambiando rápidamente entre sus disfraces, de la misma manera que se logra la animación en los dibujos animados.

1. Crea un personaje simple, por ejemplo una persona de palitos, como el de la figura 1.4.3.
2. Crea un segundo disfraz para el personaje, cambiando la posición de los brazos y piernas, simulando el caminar del personaje.
3. Crea un tercer y cuarto disfraz para el personaje, cambiando la posición de los brazos y piernas, simulando un salto del personaje.

Nota:

- Puede considerar más disfraces para el personaje para lograr una animación más fluida.
- Puede considerar videos de internet para ver cómo se logra la animación de un personaje caminando y saltando, con el fin de lograr una animación más realista.



Figura 1.5: Personaje a replicar.

Ejercicio 1.4

Repita el ejercicio anterior, pero esta vez use imágenes de una persona real, puede obtener estas imágenes a partir de fotografías de una persona sacadas con un teléfono móvil con cámara o una cámara digital.

1.5. Posicionamiento de objetos y escenarios

En Scratch, el escenario funciona como un plano cartesiano en el que los objetos o personajes pueden moverse y posicionarse. Este escenario tiene dimensiones de 480 unidades de ancho y 360 unidades de alto. Su centro se encuentra en las coordenadas $(0,0)$, por lo que el eje x se extiende desde -240 hasta 240 , mientras que el eje y se extiende desde -180 hasta 180 , como se muestra en la figura 1.6.

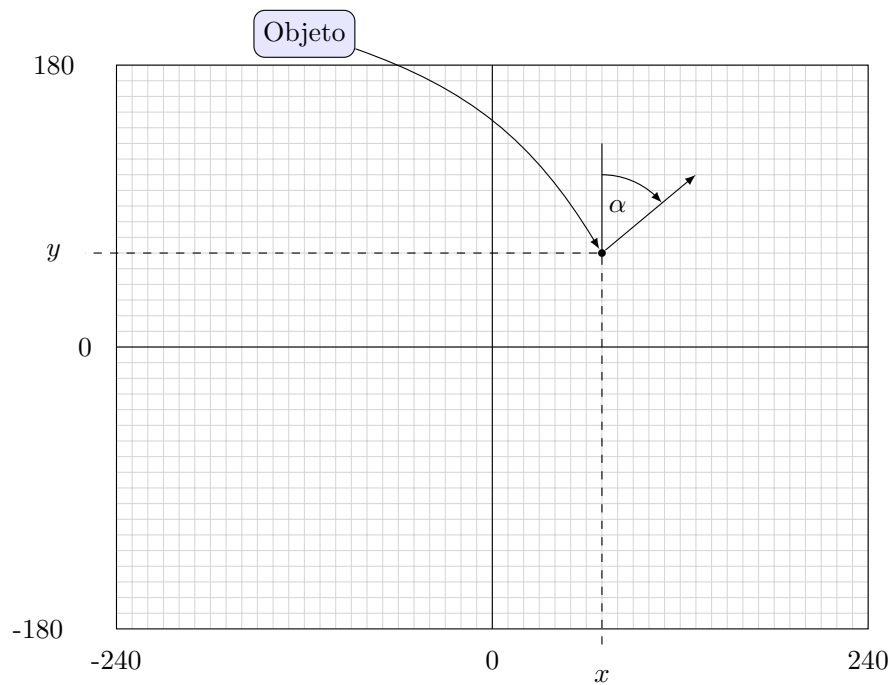


Figura 1.6: Escenario y coordenadas de un objeto en Scratch.

Claim 1.1 ¡Cuidado!

A diferencia de la convención matemática usual, donde los ángulos se miden en sentido antihorario desde el eje x positivo, en Scratch, los ángulos se miden en sentido horario desde el eje y positivo.

1.5.1. Personajes y panel de objetos

Los personajes en Scratch, también llamados *sprites*, son los objetos que se ubican y se desplazan dentro del escenario. Cada personaje aparece listado en el **panel de objetos**, ver figura 1.7, ubicado en la parte inferior de la pantalla.

Al seleccionar un personaje desde este panel, la zona de código muestra únicamente los bloques asociados a ese personaje. Además, desde el panel se pueden observar y modificar algunas de sus propiedades principales, como su nombre, posición, dirección y tamaño.

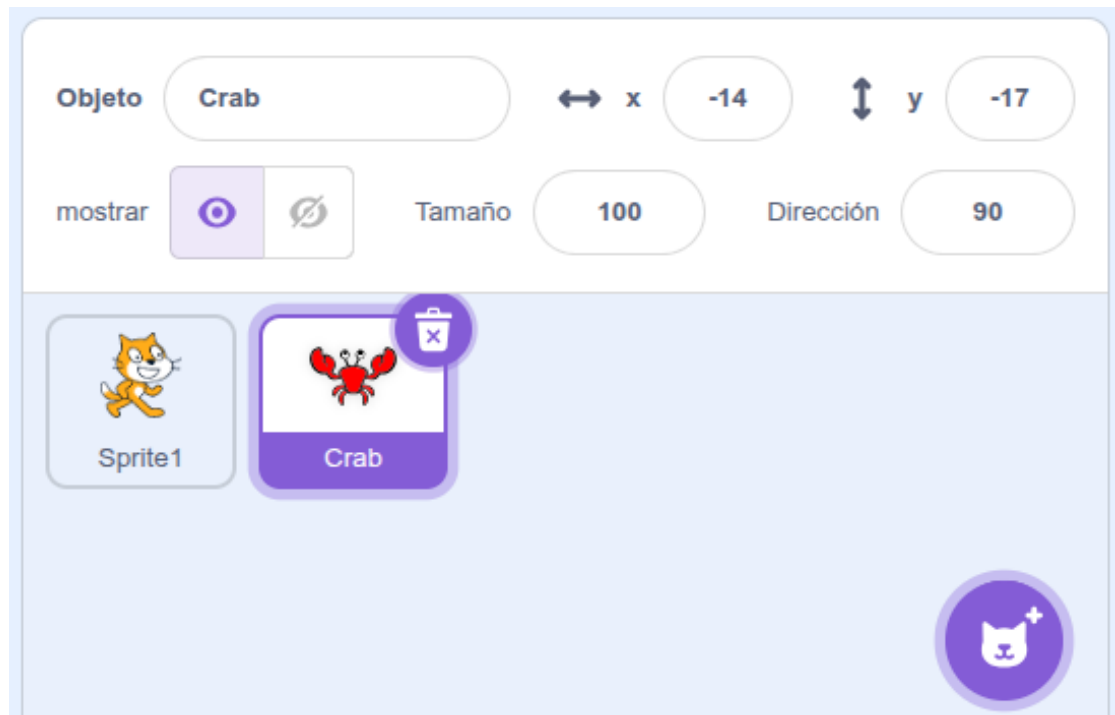


Figura 1.7: Panel de objetos en Scratch mostrando dos personajes.

1.5.2. Propiedades de un personaje

Cada personaje posee un conjunto de propiedades que permiten identificarlo, ubicarlo y modificar su apariencia dentro del escenario. Las principales son las siguientes:

- **Nombre:** permite identificar al personaje dentro del proyecto.
- **Posición:** corresponde a las coordenadas (x, y) del personaje en el escenario. Estas coordenadas pertenecen al intervalo

$$(x, y) \in [-240, 240] \times [-180, 180].$$

- **Dirección:** indica hacia dónde apunta el personaje. Esta dirección se mide en grados, desde el eje y positivo y en el sentido de las agujas del reloj. En Scratch, la dirección α puede tomar valores en el intervalo

$$\alpha \in [-180, 180[.$$

- **Tamaño:** indica el tamaño relativo del personaje respecto de su tamaño original. Por ejemplo, un valor de 100 % representa el tamaño original, mientras que un valor de 200 % indica que el personaje será el doble de grande.
- **Capa:** determina si el personaje aparece delante o detrás de otros objetos en el escenario.

1.5.3. Orientación de un personaje

La dirección de un personaje determina hacia dónde está apuntando. En Scratch, algunas direcciones importantes son:

- 90°: hacia la derecha.

- 0°: hacia arriba.
- -90°: hacia la izquierda.
- 180°: hacia abajo.

La figura 1.8 muestra el menú de dirección de un personaje en Scratch.

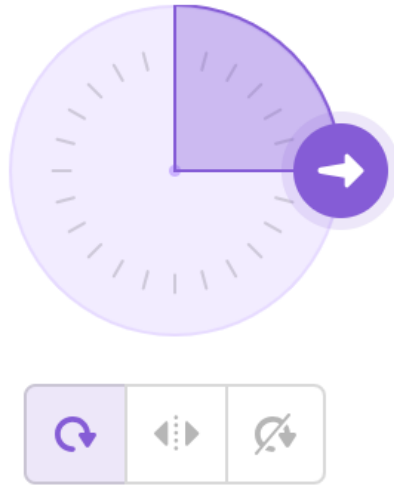


Figura 1.8: Menú de ángulo de un personaje en Scratch. En este caso, el personaje está apuntando hacia la derecha.

1.5.4. Agregar, seleccionar, duplicar y eliminar personajes

Para agregar un personaje nuevo, se debe dejar el puntero sobre el ícono **Elegir un objeto** (🐱), ubicado en la parte inferior derecha del escenario, y luego seleccionar una de las opciones disponibles.

Una vez agregado, el personaje aparecerá en el **panel de objetos**. Desde este panel se pueden realizar las siguientes acciones:

- **Seleccionar un personaje:** hacer clic sobre él en el panel de objetos.
- **Eliminar un personaje:** hacer clic en el ícono de basurero que aparece sobre el personaje, o hacer clic derecho sobre él y seleccionar la opción **Eliminar** en el menú contextual.
- **Duplicar un personaje:** hacer clic derecho sobre el personaje y seleccionar la opción **Duplicar**. Esto crea una copia exacta del personaje seleccionado.

1.5.5. Transformaciones y orden visual

Además de modificar la posición y dirección de un personaje, Scratch permite cambiar su tamaño y su orden visual dentro del escenario.

- El **tamaño** de un personaje se modifica mediante valores porcentuales. Un tamaño de 100% corresponde al tamaño original.
- Cada personaje tiene una **posición en la pila de capas**. Esto determina si aparece delante o detrás de otros objetos.
- El orden de las capas se puede modificar mediante bloques de apariencia, como *traer al frente* o *enviar atrás*.

Capítulo 2

Animaciones

Animar un personaje en Scratch consiste en cambiar su apariencia, ángulo y posición a lo largo del tiempo, creando la ilusión de movimiento. Esto se logra mediante el uso de **bloques de código** que controlan los disfraces, la posición y la dirección del personaje.

2.1. Mis primeros bloques

Los **bloques** son las piezas fundamentales de Scratch: cada uno representa una instrucción o acción que los personajes pueden ejecutar. Estas instrucciones se **arrastran** desde la **paleta de bloques** hasta el **área de código** y se conectan como si fueran piezas de rompecabezas, formando programas visuales.



Figura 2.1: Los bloques de Scratch tienen formas específicas que indican cómo se pueden conectar entre sí.

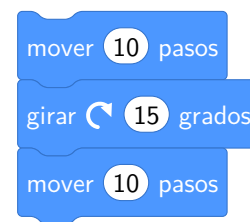


Figura 2.2: Tres bloques de movimiento conectados para mover un personaje 10 pasos, girar 15 grados y mover 10 pasos.

Ejercicio 2.1 (Ejemplo de bloques)

Para un personaje de Scratch:

- replique el código de la figura 2.1.
- Haga click en el el conjunto de tres bloques que acaba de crear. ¿Observa lo descrito en ellos?

Una de las principales ventajas de Scratch es que los bloques están organizados en categorías

por colores (movimiento, apariencia, eventos, etc.), lo que facilita su exploración y uso incluso para quienes están comenzando a programar. Los distintos colores de bloques son:

 Movimiento	Los bloques de Movimiento permiten desplazar y girar los objetos, cambiar su posición, orientarlos en una dirección determinada y hacer que se deslicen por el escenario.
 Apariencia	Los bloques de Apariencia permiten modificar la forma en que se muestran los objetos y el escenario. Con ellos se pueden cambiar disfraces, fondos, tamaños, efectos gráficos y mostrar mensajes.
 Sonido	Los bloques de Sonido permiten reproducir y detener sonidos, modificar el volumen y aplicar efectos como cambios de tono o de velocidad.
 Eventos	Los bloques de Eventos permiten iniciar programas cuando ocurre una acción determinada, como hacer clic en la bandera verde, presionar una tecla, pulsar un objeto o recibir un mensaje.
 Control	Los bloques de Control permiten organizar el flujo de un programa mediante esperas, repeticiones, condiciones, ciclos, detenciones y la creación de clones.
 Sensores	Los bloques de Sensores permiten detectar interacciones y obtener información del entorno, como el contacto entre objetos, la distancia al puntero, las teclas presionadas o la respuesta del usuario.
 Operadores	Los bloques de Operadores permiten realizar cálculos matemáticos, comparar valores, combinar condiciones lógicas y trabajar con textos y números aleatorios.
 Variables	Los bloques de Variables permiten almacenar y modificar datos durante la ejecución de un programa. También permiten crear listas para guardar varios valores de manera ordenada.
 Mis bloques	La categoría Mis bloques permite crear bloques personalizados para agrupar instrucciones, reutilizar código y dividir un programa en partes más pequeñas y fáciles de comprender.

Claim 2.1.1 Bloques y personajes

Cada personaje tiene su propio conjunto de bloques, y los bloques de un personaje no afectan a otros personajes a menos que se utilicen bloques de comunicación o eventos.

Es posible identificar a qué personaje pertenece un bloque pues aparece un icono del personaje en la esquina superior derecha del bloque y además está seleccionado el personaje en el panel de objetos (borde morado).

2.1.1. El bloque

El bloque “**al presionar bandera verde**” es uno de los más utilizados. Su función es **indicar el inicio del programa**, activando todas las secuencias de bloques que estén conectadas a él.

- Cada sprite puede tener uno o más bloques de evento.
- Al presionar la bandera verde, todos los bloques de evento que comienzan con este bloque se ejecutan **en paralelo**.
- Es recomendable usarlo para:
 - Posicionar los personajes en su lugar inicial.
 - Reiniciar variables y puntajes.
 - Cambiar el fondo inicial.
 - Iniciar animaciones o sonidos.

Este bloque se encuentra en la categoría “*Eventos*” y se reconoce fácilmente por su forma de sombrero (no puede conectarse arriba de él).

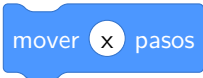
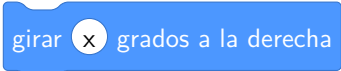
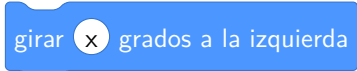


Figura 2.3: Bloque “al presionar ”

2.1.2. Bloques de

Los bloques de movimiento están en la paleta de bloques indicados con color azul  y permiten **mover y posicionar personajes en el escenario**. Trabajan principalmente sobre las coordenadas (x, y) y la *dirección* en la que apunta cada sprite.

Alguno de estos bloques son:

-  desplaza al sprite en la dirección actual.
-  o  cambia su orientación hacia la izquierda o derecha.

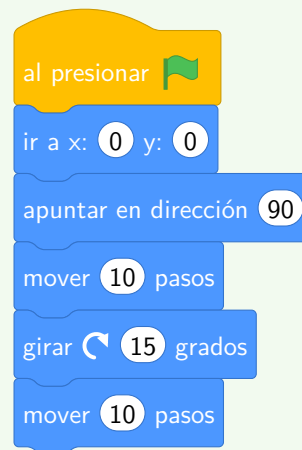
- **ir a x: x y: y** posiciona al sprite en una ubicación exacta.
- **deslizar t s a x: x y: y** mueve al sprite de forma suave.
- **rebotar si toca un borde** evita que desaparezca del escenario al llegar a un borde.
- **fijar x a $valor$** o **fijar y a $valor$** asigna un valor a la coordenada x o y (inicializa o reemplaza su contenido).
- **cambiar x en $valor$** o **cambiar y en $valor$** incrementa o decrementa la coordenada x o y en la cantidad indicada, puede ser un número negativo para moverse en la dirección opuesta.

Estos bloques se pueden combinar para crear trayectorias, animaciones de movimiento y efectos visuales interesantes.

Ejemplo 2.1

El código de la derecha realiza las siguientes acciones al presionar la bandera verde:

1. **inicializa** la posición del personaje en el origen,
2. **apunta** a la derecha,
3. **mueve** 10 pasos hacia adelante,
4. **gira** 15 grados en dirección de las agujas del reloj,
5. **mueve** otros 10 pasos.



2.1.3. Bloques de **apariciencia**

Los bloques de apariencia están en la paleta de bloques indicados con color púrpura ■ y permiten cambiar la forma en la que se ven los personajes. Principalmente permiten elegir los **disfraces** que usa cada personaje. Los **disfraces** son **versiones gráficas diferentes de un mismo personaje** y se puede acceder a ellos desde la pestaña de “disfraces”. Cambiar disfraces permite crear animaciones simples, como caminar, correr o hablar, sin necesidad de código complejo.

- **cambiar disfraz a $disfraz1$** reemplaza el disfraz actual por otro específico.
- **siguiente disfraz** cambia automáticamente al siguiente disfraz de la lista.

- **mostrar** y **esconder** hacen visible o invisible al personaje en el escenario.
- **cambiar tamaño por 10 %** y **fijar tamaño a 100 %** modifican la escala del personaje.
- **ir a capa de adelante** y **ir 1 capas hacia adelante** mueve al personaje entre capas hacia adelante o atrás.

Claim 2.1.2 Inicialización

Al clicar **al presionar** por segunda vez, el programa se ejecuta desde el principio, pero los personajes pueden estar en posiciones o estados diferentes a los de la primera ejecución. Esto puede generar comportamientos inesperados en la animación. Al proceso de establecer un estado inicial para los personajes y variables se le llama **inicialización**.

- Es recomendable establecer la posición inicial del personaje al comienzo del programa.
- Esto permite que cada ejecución inicie siempre desde la misma ubicación, evitando comportamientos aleatorios no deseados.
- Esto se logra comúnmente usando un bloque de evento, por ejemplo, **al presionar**, combinado con bloques como **ir a x: 0 y: 0**, **apuntar en dirección 90**, **cambiar disfraz a disfraz1**, etcétera.

Ejercicio 2.2

Haga una inicialización de un personaje tal que:

- Apunte hacia la derecha.
- Se ubique en la posición $x = 0, y = 0$.
- Cambie al disfraz1.
- Se muestre en el escenario.
- Se fije el tamaño al 100 %.

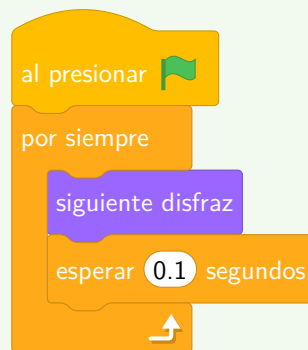
2.1.4. Bloques de control I

Los bloques de control están en la paleta de bloques indicados con color naranja y permiten gestionar el flujo de ejecución del programa. Para crear animaciones serán suficientes los siguientes bloques:

- **esperar t segundos** pausa la ejecución durante el tiempo indicado.
- **por siempre** repite indefinidamente los bloques dentro de él.
- **repetir n** repite un número específico de veces los bloques dentro de él.

Ejemplo 2.2

Este pequeño programa hace que el personaje cambie de disfraz continuamente, generando un efecto de movimiento fluido. Al combinar estos bloques con los de movimiento, los estudiantes pueden crear animaciones básicas de caminar, correr o flotar.



Ejercicio 2.3

Implemente el código del ejemplo anterior en el Gato Scratch (personaje por defecto) y observe cómo cambia de disfraz continuamente. Ajuste el tiempo de espera para ver cómo afecta la velocidad del cambio de disfraces.

Claim 2.1

Los bloques de control anteriores tienen las siguientes aplicaciones al momento de realizar animaciones:

- esperar t segundos** : Permite controlar el ritmo de una animación, establecer pausas entre cambios de disfraz y coordinar las acciones de distintos personajes.
- por siempre** : Permite crear animaciones continuas, como hacer que un personaje camine, cambie de disfraz o se mueva mientras el programa esté en ejecución.



Permite repetir una secuencia de animación un número determinado de veces, como hacer que un personaje salte, avance o cambie de disfraz antes de continuar con otra acción.

2.1.5. Bloques de **sonido** *

Los bloques de sonido están en la paleta de bloques indicados con color lila ■ y permiten agregar, reproducir y modificar sonidos dentro de un proyecto. Estos bloques son útiles para acompañar animaciones, crear efectos, hacer que un personaje hable mediante sonidos o agregar música de fondo.

Algunos de estos bloques son:

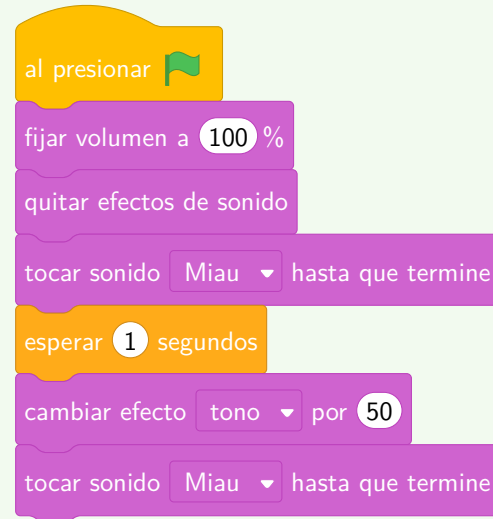
- **tocar sonido** Miau ▼ **hasta que termine** reproduce un sonido y espera a que termine antes de continuar con el siguiente bloque.
- **iniciar sonido** Miau ▼ reproduce un sonido y continúa inmediatamente con los siguientes bloques, sin esperar a que el sonido termine.
- **detener todos los sonidos** detiene todos los sonidos que se estén reproduciendo en ese momento.
- **cambiar volumen por** -10 aumenta o disminuye el volumen del sprite. Si el número es negativo, el volumen disminuye.
- **fijar volumen a** 100 % establece el volumen del sprite en un valor específico.
- **cambiar efecto** tono ▼ **por** 10 modifica un efecto de sonido, por ejemplo el tono, haciendo que el sonido se escuche más agudo o más grave.
- **establecer efecto** tono ▼ **a** 100 fija un efecto de sonido en un valor determinado.
- **quitar efectos de sonido** elimina los efectos aplicados al sonido y lo deja en su estado original.

Estos bloques se pueden combinar con los bloques de movimiento, apariencia y control para crear animaciones más expresivas. Por ejemplo, un personaje puede moverse, cambiar de disfraz y reproducir un sonido cada vez que realiza una acción.

Ejemplo 2.3

El código de la derecha realiza las siguientes acciones al clicar sobre **al presionar**  :

1. **inicializa** el volumen del personaje al 100 %,
2. **quita** cualquier efecto de sonido aplicado anteriormente,
3. **reproduce** el sonido seleccionado,
4. **espera** un segundo,
5. **cambia** el tono del sonido,
6. **reproduce** nuevamente el sonido, pero ahora con un efecto aplicado.



Claim 2.1.3 Sonidos e inicialización

Al igual que ocurre con la posición, el disfraz o el tamaño de un personaje, los sonidos también pueden quedar en un estado distinto después de ejecutar un programa. Por ejemplo, el volumen puede quedar bajo o un efecto de tono puede seguir activo.

Por esta razón, es recomendable inicializar los sonidos al comenzar el programa. Esto se puede hacer usando bloques como: **fijar volumen a 100 %** y **quitar efectos de sonido**.

Ejercicio 2.4

Haga una inicialización de sonido para un personaje tal que:

- a) El volumen se fije al 100 %.
- b) Se quiten todos los efectos de sonido.
- c) Al presionar la bandera verde, el personaje reproduzca un sonido.
- d) Después de reproducir el sonido, espere 1 segundo.
- e) Finalmente, detenga todos los sonidos.

2.2. El escenario: bloques y fondos

El **escenario** es el espacio en el que se desarrolla un proyecto de Scratch. En él aparecen los personajes, se muestran las animaciones y se representan los distintos lugares o ambientes de una historia.

Al igual que los personajes, el escenario puede tener sus propios bloques de código. Para

programarlo, se debe seleccionar el escenario en el panel de objetos. Al hacerlo, la paleta mostrará únicamente los bloques que pueden ser utilizados por este elemento.

Claim 2.2.1 Escenario y personajes

El escenario no es un personaje y, por lo tanto, tiene algunas características diferentes:

- No puede desplazarse por el sistema de coordenadas.
- No posee bloques de movimiento.
- No utiliza disfraces, sino **fondos**.
- No puede mostrarse ni esconderse.
- Puede ejecutar programas, reproducir sonidos y enviar o recibir eventos.

Los bloques agregados al escenario pertenecen únicamente a este. Para modificar el comportamiento de un personaje, se debe seleccionar dicho personaje y programarlo por separado.

2.2.1. Los fondos

Los **fondos** son las diferentes imágenes que puede mostrar el escenario. Cumplen una función similar a la de los disfraces de los personajes: permiten cambiar la apariencia del proyecto sin reemplazar el escenario.

Los fondos se pueden agregar, dibujar o modificar desde la pestaña “Fondos”. Un proyecto puede contener varios fondos, por ejemplo:

- un paisaje de día y otro de noche,
- diferentes habitaciones de una casa,
- las distintas pantallas de un videojuego,
- lugares diferentes dentro de una historia,
- una portada, una pantalla de instrucciones y una pantalla final.

Cada fondo tiene un nombre. Es recomendable utilizar nombres que permitan reconocer fácilmente su función, como *Portada*, *Bosque*, *Noche* o *Fin*, en lugar de conservar nombres poco descriptivos.

2.2.2. Bloques de **apariencia del escenario**

Cuando el escenario está seleccionado, la categoría Apariencia contiene bloques específicos para trabajar con fondos. Algunos de estos bloques son:

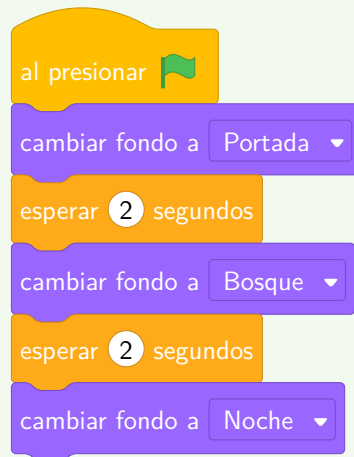
- **cambiar fondo a fondo1** ; reemplaza inmediatamente el fondo actual por el fondo seleccionado.
- **cambiar fondo a fondo1 y esperar** ; cambia el fondo y espera a que terminen los programas iniciados por ese cambio antes de continuar.

- **siguiente fondo**; muestra el fondo que sigue en la lista. Si se utiliza estando seleccionado el último fondo, Scratch vuelve al primero.
- **cambiar efecto** **color** por **25**; modifica progresivamente un efecto gráfico del escenario.
- **fijar efecto** **color** a **0**; asigna un valor específico al efecto gráfico seleccionado.
- **quitar efectos gráficos**; elimina todos los efectos aplicados al fondo y recupera su apariencia original.

Los bloques de cambio de fondo pueden combinarse con bloques de control para crear una secuencia de escenas.

Ejemplo 2.4

El siguiente programa muestra tres fondos de manera consecutiva. Cada fondo permanece visible durante dos segundos antes de mostrar el siguiente.



Al presionar la bandera verde, el escenario realiza las siguientes acciones:

1. muestra el fondo llamado *Portada*,
2. espera dos segundos,
3. cambia al fondo llamado *Bosque*,
4. espera otros dos segundos,
5. finalmente, muestra el fondo llamado *Noche*.

Ejercicio 2.5

Agregue al escenario al menos tres fondos diferentes y realice un programa que:

- a) Comience al presionar la bandera verde.

- b) Muestre el primer fondo.
- c) Espere un segundo.
- d) Muestre el segundo fondo.
- e) Espere dos segundos.
- f) Muestre el tercer fondo.

Modifique posteriormente los tiempos de espera y observe cómo cambia la velocidad de la secuencia.

2.2.3. Fondos y bloques de control

El bloque **siguiente fondo** puede colocarse dentro de un ciclo para cambiar el escenario repetidamente. Por ejemplo:



Este programa cambia seis veces el fondo, dejando pasar un segundo entre cada cambio. Si el proyecto tiene menos de seis fondos, Scratch vuelve al primero después de mostrar el último.

También es posible utilizar un ciclo infinito:



En este caso, los fondos cambian continuamente hasta que se detiene el programa.

Claim 2.2.2 Cambios demasiado rápidos

Si se utiliza el bloque **siguiente fondo** dentro de un ciclo sin agregar un bloque de espera,

los fondos pueden cambiar tan rápidamente que será difícil distinguirlos.

Para controlar la velocidad de la animación, se recomienda colocar un bloque

esperar **t** segundos después de cada cambio de fondo.

2.2.4. Eventos asociados a los fondos

Un cambio de fondo también puede iniciar programas. Para ello se utiliza el bloque de evento:

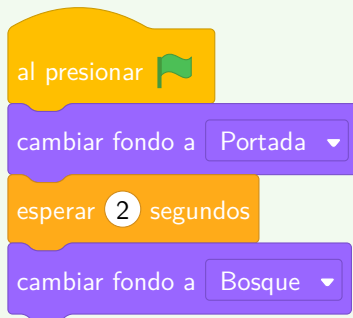
cuando el fondo cambie a **Bosque**

Este bloque ejecuta la secuencia conectada debajo de él cuando el escenario comienza a mostrar el fondo seleccionado. Puede utilizarse tanto en el escenario como en los personajes.

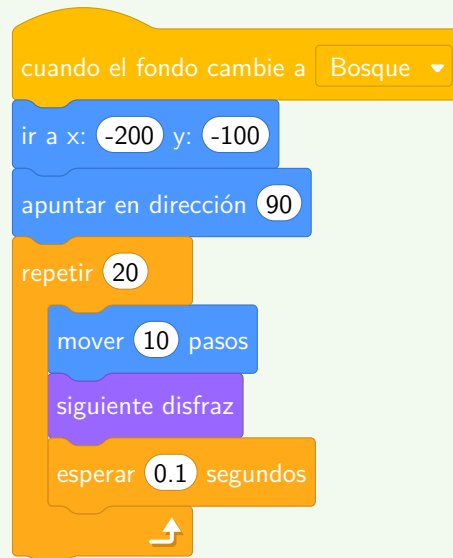
Por ejemplo, el escenario puede cambiar al fondo *Bosque* y un personaje puede comenzar a caminar cuando detecta ese cambio.

Ejemplo 2.5

Código del escenario



Código del personaje



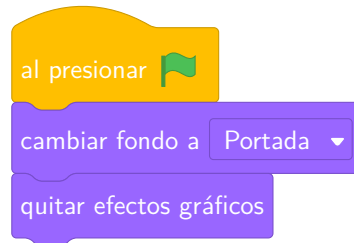
En este ejemplo:

1. el escenario comienza mostrando la portada,
2. después de dos segundos cambia al fondo *Bosque*,
3. el cambio de fondo activa el programa del personaje,
4. el personaje se ubica en su posición inicial,
5. finalmente, se desplaza mientras cambia de disfraz.

De esta forma, los fondos no solo modifican la apariencia del proyecto, sino que también pueden utilizarse para coordinar las acciones de los personajes.

2.2.5. Inicialización del escenario

Al ejecutar varias veces un proyecto, el escenario puede quedar mostrando un fondo diferente o conservar efectos gráficos aplicados durante la ejecución anterior. Para obtener siempre el mismo resultado, es recomendable inicializarlo al presionar la bandera verde.



Este programa establece el fondo inicial y elimina cualquier efecto gráfico aplicado anteriormente.

Claim 2.2.3 Inicialización del fondo

El fondo que aparece al comenzar un proyecto no depende necesariamente de su posición en la lista. El escenario puede conservar el último fondo mostrado antes de detener el programa.

Por esta razón, conviene incluir siempre un bloque como  al comienzo del programa, especialmente cuando los fondos representan escenas, niveles o pantallas diferentes.

Ejercicio 2.6

Cree un proyecto con los fondos *Portada*, *Día*, *Noche* y *Fin*. Luego programe el escenario y un personaje para que:

- Al presionar la bandera verde, el escenario muestre la portada y quite los efectos gráficos.
- Después de dos segundos, el escenario cambie al fondo *Día*.
- Cuando aparezca el fondo *Día*, el personaje se muestre y comience a caminar.
- Después de tres segundos, el escenario cambie al fondo *Noche*.
- Al aparecer el fondo *Noche*, se aplique un efecto gráfico al escenario.
- Finalmente, el escenario muestre el fondo *Fin* y el personaje se esconda.

Capítulo 3

Videojuegos

A diferencia de las animaciones, los videojuegos no se desarrollan de manera completamente automática. En ellos, el jugador participa activamente, controla personajes, toma decisiones y modifica lo que ocurre en la pantalla. Por esta razón, además de movimiento, apariencia y sonido, un videojuego necesita mecanismos que permitan detectar las acciones del jugador, las interacciones entre los personajes y responder a ellas.

En Scratch, estas acciones pueden realizarse mediante el teclado, el ratón o distintos sensores. Por ejemplo, un personaje puede moverse al presionar las flechas del teclado, saltar al pulsar la barra espaciadora o seguir la posición del puntero del ratón. También es posible detectar colisiones, verificar si un personaje toca un objeto o determinar si una tecla está siendo presionada.

3.1. Interacción con el jugador

En un videojuego, el jugador no solo observa lo que ocurre: también toma decisiones. En Scratch, esa interacción se programa mediante eventos, sensores y bloques de movimiento que permiten responder al teclado y al ratón.

3.1.1. Bloques de y

Muchas veces el programa debe *decidir* qué hacer: mover un personaje solo si se presiona una tecla, sumar puntos si se toca una moneda, perder una vida si aparece una colisión o terminar la partida cuando se cumple una meta. Para programar estas decisiones se usan condiciones.

Una condición es una pregunta que Scratch puede responder con solo dos valores posibles:

- **Verdadero:** la condición se cumple.
- **Falso:** la condición no se cumple.

A este tipo de respuestas se les llama *booleanas*. Por eso, los bloques que representan preguntas de verdadero o falso se llaman *bloques booleanos*. En Scratch se reconocen porque tienen forma hexagonal y pueden colocarse dentro de bloques de control.



Por ejemplo, la pregunta  es verdadera mientras el jugador mantiene presionada la barra espaciadora, y falsa cuando no la está presionando.

Booleanos en sensores y operadores

En Scratch hay varios bloques que producen respuestas booleanas. Algunos pertenecen a *Sensores* y permiten preguntar por acciones del jugador o contactos entre objetos:

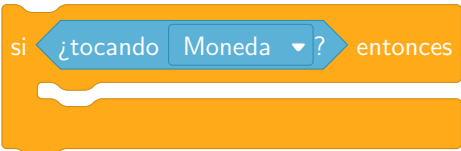
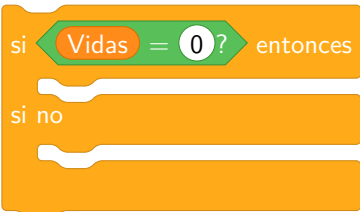
-  pregunta si una tecla está siendo presionada.
-  pregunta si el personaje actual está tocando a otro objeto.
-  pregunta si el jugador está haciendo clic.

Otros booleanos se construyen con *Operadores*. Estos permiten comparar números, variables o combinar varias condiciones:

-  pregunta si el puntaje es exactamente 10.
-  pregunta si quedan menos de una vida.
-  exige que dos condiciones sean verdaderas al mismo tiempo.
-  se cumple si al menos una de las dos condiciones es verdadera.
-  invierte la respuesta de una condición.

Estructuras de control que dependen de booleanos

Los bloques de control usan condiciones booleanas para decidir cuándo ejecutar una parte del programa. Los más importantes para videojuegos son:

-  ejecuta instrucciones solamente cuando la condición es verdadera.
-  elige entre dos caminos: uno si la condición es verdadera y otro si es falsa.
-  detiene momentáneamente el programa hasta que la condición se vuelva verdadera.

- repetir hasta que repite instrucciones mientras la condición sea falsa, y se detiene cuando se vuelve verdadera.

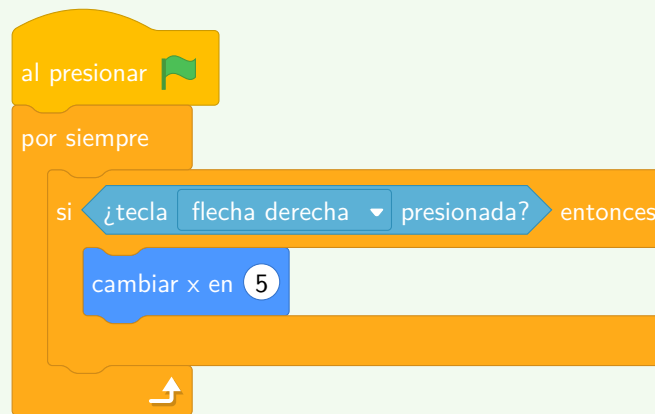
Claim 3.1.1 Preguntas dentro de ciclos

En un videojuego las condiciones suelen revisarse muchas veces. Por eso es común poner bloques *si... entonces* dentro de un ciclo *por siempre*. Así el programa puede detectar continuamente teclas, colisiones, puntaje, vidas o estados de victoria.

Ejemplos simples

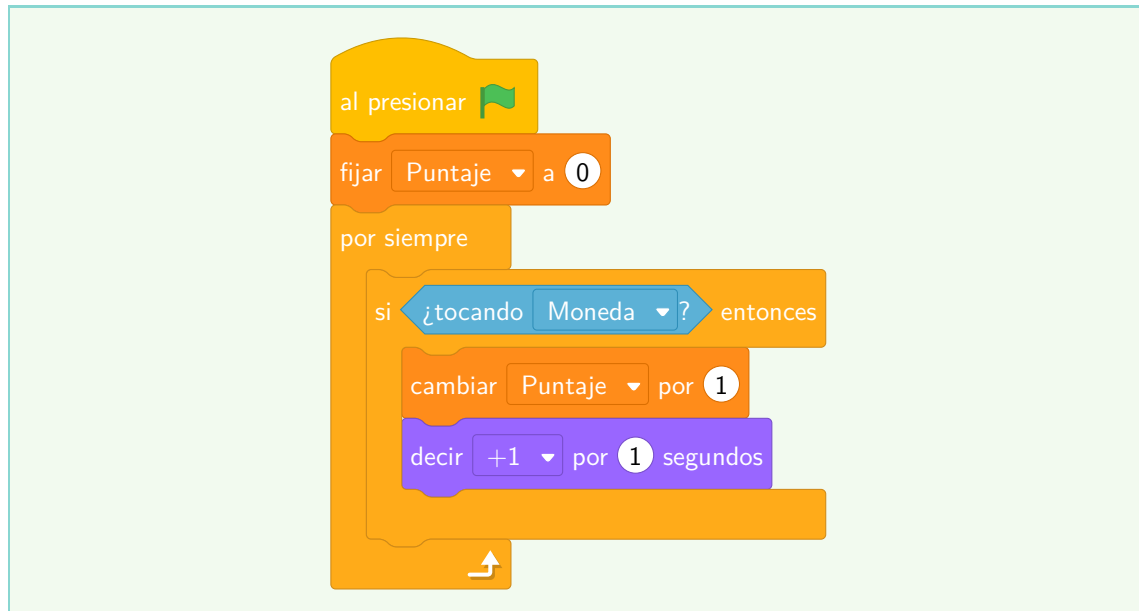
Ejemplo 3.1 (Mover al jugador solo si se presiona una tecla)

Este programa pregunta todo el tiempo si la flecha derecha está presionada. Si la respuesta es verdadera, el personaje avanza.

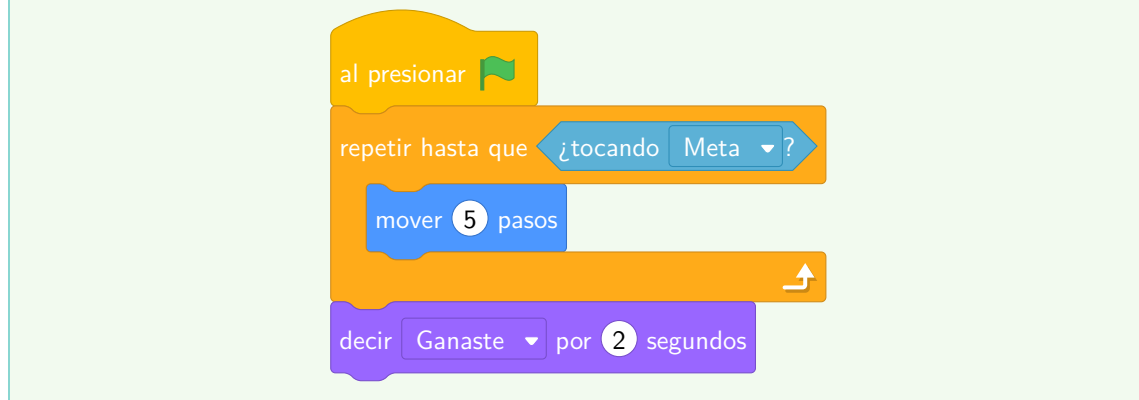


Ejemplo 3.2 (Recoger una moneda)

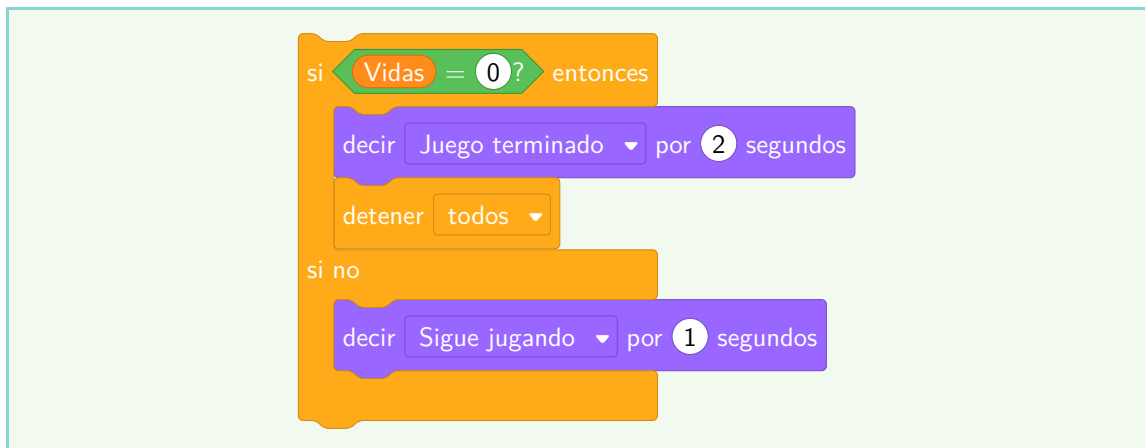
La condición permite detectar una colisión. Cuando el jugador toca la moneda, el puntaje aumenta.

**Ejemplo 3.3** (Ganar al llegar a una meta)

El bloque *repetir hasta que* mantiene activo el movimiento del personaje hasta que se cumple la condición de llegada.

**Ejemplo 3.4** (Perder si no quedan vidas)

Una comparación también es un booleano. En este caso, el juego revisa si la variable *Vidas* llegó a cero.



3.1.2. Eventos de teclado

El bloque **al presionar tecla espacio** inicia un programa cuando el usuario presiona una tecla específica. Se encuentra en la categoría *Eventos* y permite asociar acciones a teclas como las flechas, la barra espaciadora o letras del teclado.

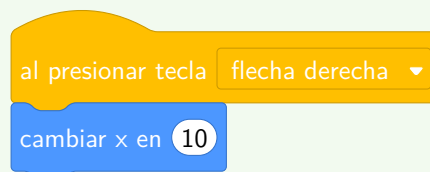
- Una tecla puede mover un personaje.
- Distintas teclas pueden activar acciones diferentes.
- Varios bloques de evento pueden pertenecer al mismo personaje.
- Si dos teclas controlan acciones opuestas, conviene revisar que no produzcan movimientos contradictorios.

Claim 3.1

El bloque **al presionar tecla espacio** es un bloque de inicio (se nota por la curvar en la parte superior), es decir no puede conectarse por arriba con otros bloques.

Ejemplo 3.5

El siguiente código mueve un personaje hacia la derecha cada vez que se presiona la flecha derecha.



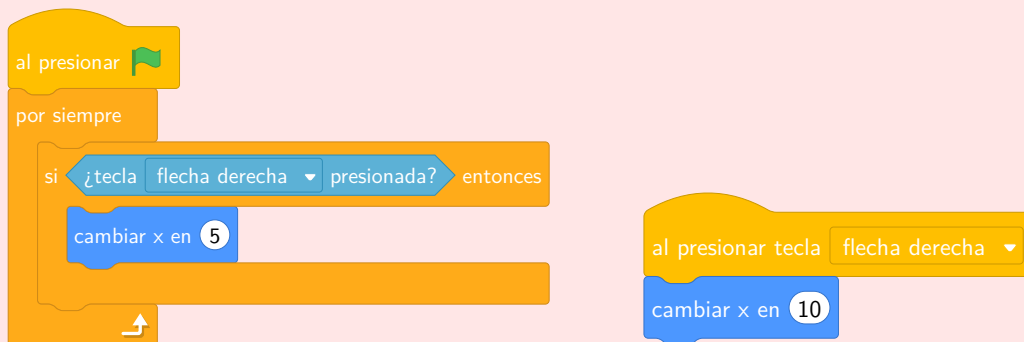
Movimiento mediante flechas

Para controlar un personaje en las cuatro direcciones principales, se pueden crear cuatro programas independientes:

**Claim 3.1.2 Presionar y mantener presionada**

El evento *al presionar tecla* se activa cuando Scratch detecta la pulsación de una tecla. En cambio, el sensor *¿tecla flecha derecha presionada?* permite preguntar continuamente si una tecla está siendo mantenida presionada.

Por esta razón, para movimientos más suaves suele usarse un ciclo *por siempre* junto con sensores de teclado.



Mueve a la derecha al presionar Flecha Derecha, mientras esté presionada Flecha Derecha seguirá moviendo a la derecha.

Mueve una vez al presionar Flecha Derecha, si se mantiene presionado no vuelve a mover.

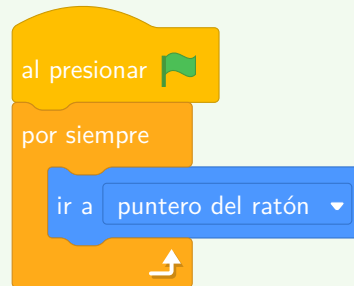
3.1.3. Uso del ratón

Scratch también permite usar el ratón como forma de entrada. El programa puede conocer la posición del puntero, hacer que un personaje lo siga o detectar clics sobre un objeto.

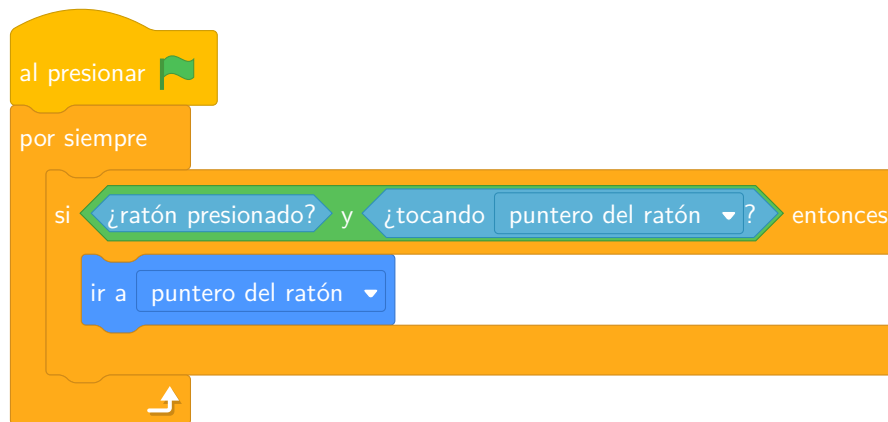
- *posición x del ratón* y *posición y del ratón* entregan las coordenadas del puntero.
- *ir a puntero del ratón* mueve el personaje hasta la posición del puntero.
- *al hacer clic en este objeto* ejecuta una acción cuando el jugador hace clic sobre el personaje seleccionado.
- *¿ratón presionado?* permite detectar si el botón del ratón está siendo presionado.

Ejemplo 3.6

Este programa hace que un personaje siga continuamente al puntero.

**Arrastrar objetos**

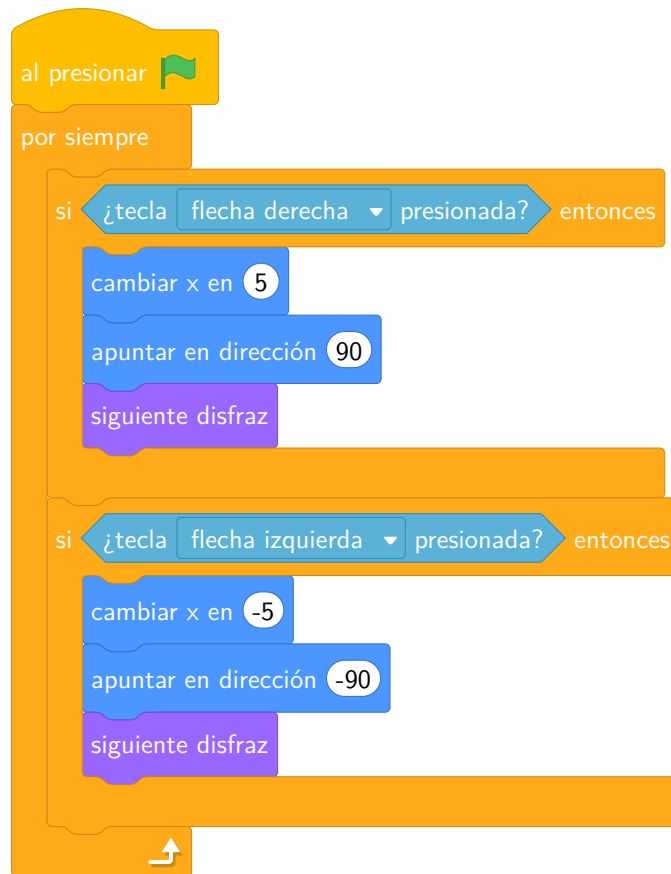
Para simular que un objeto puede ser arrastrado, se puede usar un ciclo que lo envíe al puntero mientras el ratón está presionado y el puntero toca el objeto.

**3.1.4. Movimiento controlado**

El movimiento controlado combina entradas del jugador, movimiento horizontal y vertical, límites del escenario, dirección y disfraces.

Movimiento horizontal y vertical

Una forma cómoda de controlar un personaje es revisar las teclas dentro de un ciclo:



Restricción dentro del escenario

El escenario tiene límites aproximados $x \in [-240, 240]$ e $y \in [-180, 180]$. Para evitar que un personaje salga, se puede revisar su posición o usar el bloque:

rebotar si toca un borde

En juegos donde el personaje no debe rebotar, sino detenerse, se recomienda controlar el movimiento con condiciones sobre las coordenadas.

Ejercicio 3.1 (Actividad: controlar un personaje mediante el teclado)

Cree un personaje controlado por el teclado que cumpla lo siguiente:

- Al presionar la bandera verde, se ubique en $x = 0, y = 0$.
- Se mueva con las cuatro flechas.
- Cambie de dirección al moverse hacia la izquierda o derecha.
- Cambie de disfraz mientras camina.
- No desaparezca al llegar a los bordes del escenario.

3.2. Sensores y colisiones

Las *colisiones* ocurren cuando dos personajes se tocan. En videojuegos, las colisiones permiten recoger premios, recibir daño, activar una meta o impedir que el jugador atraviese una pared. Son parte importante de la interactividad de los videojuegos, y se programan mediante sensores y condiciones.

Los *sensores* permiten que un programa conozca lo que está ocurriendo en el escenario. Con ellos se pueden detectar teclas, colores, distancias y contactos entre personajes. Al combinarlos con condiciones, Scratch puede tomar decisiones durante el juego.

3.2.1. Bloques de sensores

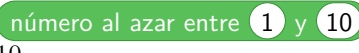
Los bloques de sensores se encuentran en la categoría celeste . Algunos de los más usados en videojuegos son:

-  detecta si el personaje toca otro objeto, el borde o el puntero del ratón.
-  detecta contacto con un color específico.
-  detecta si una tecla está siendo presionada.
-  entrega la distancia entre el personaje actual y otro elemento.
-  y  entregan la posición del puntero.

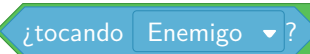
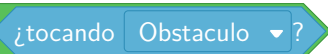
3.2.2. Operadores

Los operadores permiten tomar transformar uno o dos valores en otro valor. Se dividen en dos grupos:

Bloques de valor: Devuelven un valor, normalmente un número o texto. Por ejemplo

-  devuelve un número aleatorio comprendido entre 1 y 10.
-  devuelve la coordenada horizontal actual del puntero del ratón.
-  devuelve la distancia entre el objeto actual y el objeto seleccionado.
-  devuelve el resultado de una operación aritmética, en este caso 15.
-  devuelve la cantidad de caracteres de un texto, en este caso 4.

Bloques booleanos: Devuelven un valor booleano, es decir, verdadero o falso. Por ejemplo

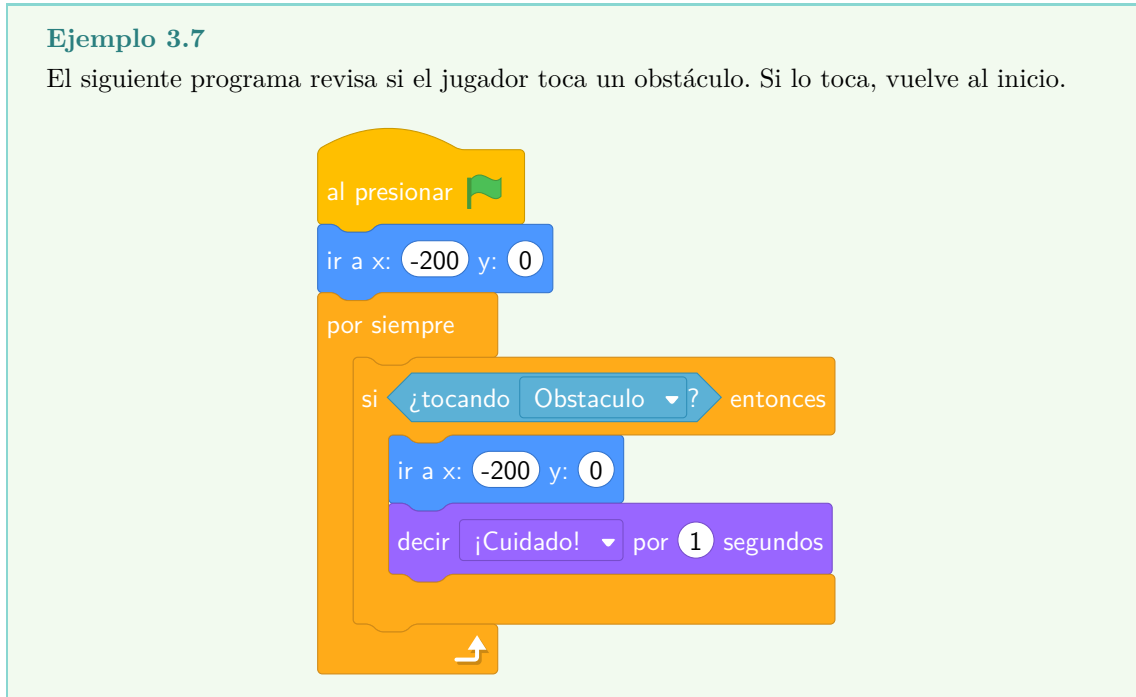
- ,  y  comparan valores.
-  y  exige que ambas condiciones sean verdaderas.
-  o  se cumple si al menos una condición es verdadera.
-  invierte el resultado de una condición.

3.2.3. Colisiones

Una colisión ocurre cuando dos elementos se tocan. En videojuegos, las colisiones permiten recoger premios, recibir daño, activar una meta o impedir que el jugador atraviese una pared.

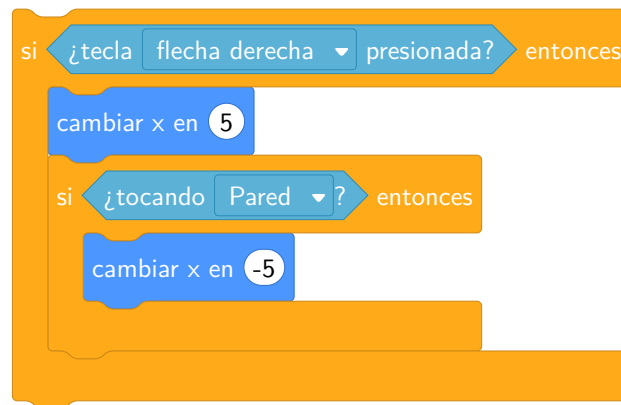
Ejemplo 3.7

El siguiente programa revisa si el jugador toca un obstáculo. Si lo toca, vuelve al inicio.



Evitar atravesar objetos

Una técnica simple para impedir que un personaje atraviese una pared consiste en moverlo, revisar si tocó la pared y, si ocurrió, deshacer el movimiento.



Ejercicio 3.2 (Actividad: crear un laberinto sencillo)

Dibuje un fondo con paredes y cree un personaje que se mueva con las flechas. El juego debe cumplir:

- El personaje parte siempre desde una posición inicial.
- Si toca una pared, no puede atravesarla.

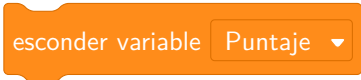
- c) Si toca la meta, muestra un mensaje de victoria.
- d) Si toca un obstáculo, vuelve al inicio.

3.3. Variables y reglas de juego

Una variable es un espacio donde Scratch guarda información que puede cambiar durante la ejecución. En videojuegos, las variables permiten controlar puntaje, vidas, energía, tiempo, niveles y otras reglas.

3.3.1. Introducción a las variables

Las variables se crean desde la categoría *Variables*. Conviene usar nombres claros como *Puntaje*, *Vidas*, *Tiempo* o *Nivel*.

-  asigna un valor inicial.
-  aumenta o disminuye su valor.
-  y  controlan si la variable aparece en el escenario.
-  entrega el valor de la variable Puntaje.

Claim 3.3.1 Inicializar variables

Toda variable importante debe fijarse al comenzar una partida. Si no se inicializa, puede conservar el valor que tenía al detener el programa anterior.

En general, es recomendable dejar esa inicialización en los códigos de bloque del escenario.

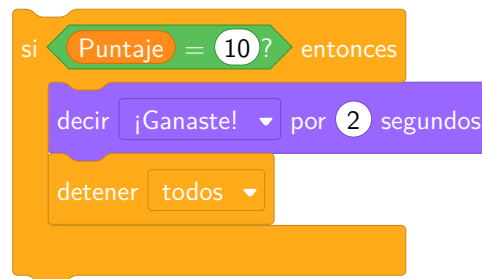
3.3.2. Puntaje

El puntaje representa el avance del jugador. Por ejemplo, puede aumentar al recoger objetos y disminuir al tocar obstáculos.



Condiciones para ganar

Una condición de victoria puede depender del puntaje:

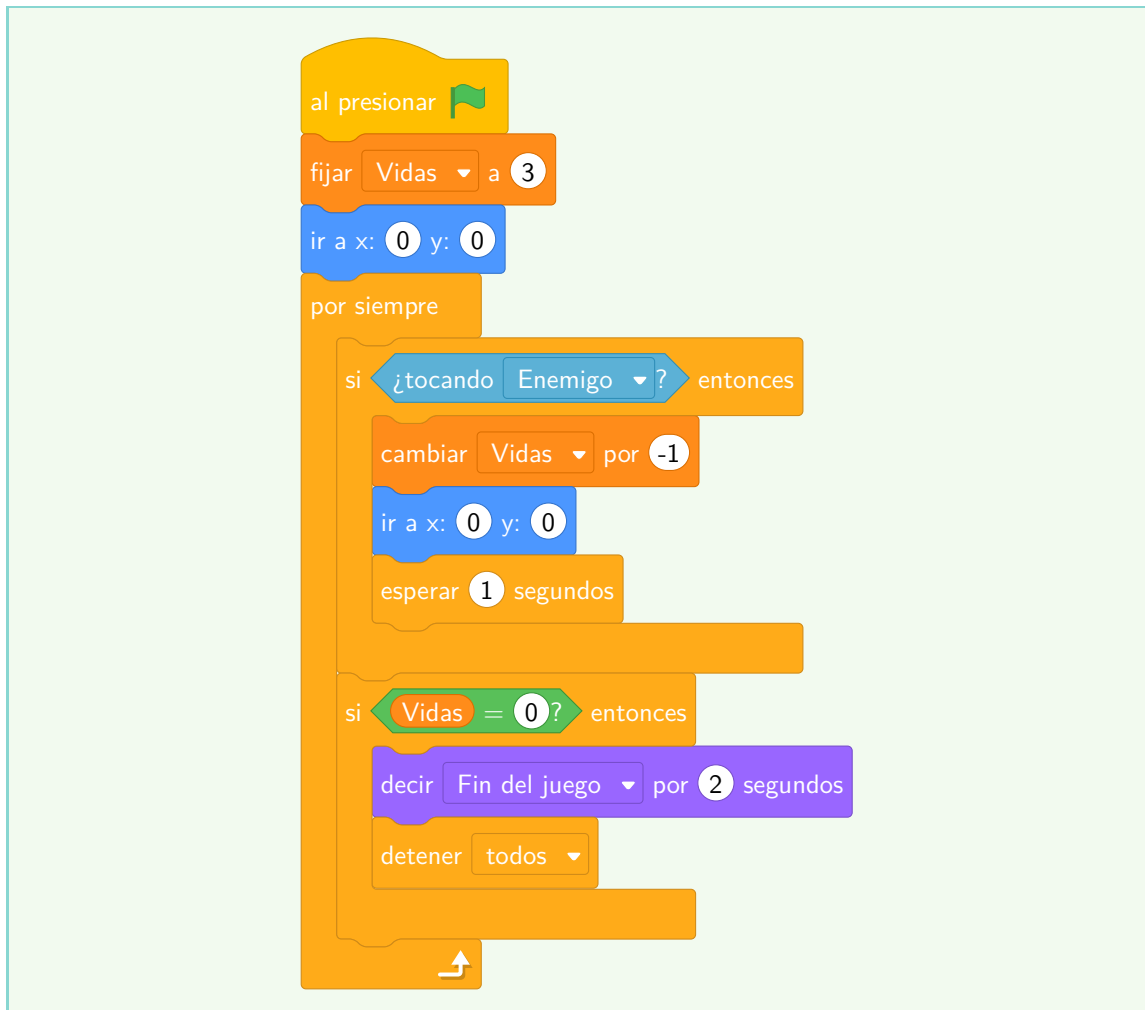


3.3.3. Vidas y energía

Las vidas permiten representar cuántas oportunidades tiene el jugador antes de perder.

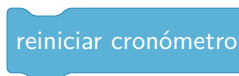
Ejemplo 3.8

Este programa resta una vida al tocar un enemigo y reinicia la posición del jugador.



3.3.4. Temporizadores

Scratch tiene un cronómetro interno que comienza a contar cuando se inicia el proyecto. Puede reiniciarse con:



También se puede crear una cuenta regresiva usando una variable.

**Ejercicio 3.3** (Actividad: juego de recolección)

Cree un juego donde el personaje deba recoger objetos antes de que se acabe el tiempo.

- Cree las variables *Puntaje*, *Vidas* y *Tiempo*.
- Inicialice sus valores al presionar la bandera verde.
- Aumente el puntaje al tocar premios.
- Reste una vida al tocar obstáculos.
- Termine la partida si el tiempo llega a 0 o si las vidas llegan a 0.
- Muestre un mensaje de victoria si el puntaje alcanza la meta.

3.4. Azar y movimiento automático

El azar permite que un juego sea menos predecible. En Scratch, los números aleatorios pueden usarse para elegir posiciones, velocidades, tiempos de aparición o direcciones de movimiento.

3.4.1. Números aleatorios

El bloque **número al azar entre 1 y 10** entrega un número dentro del intervalo indicado. Cada vez que se ejecuta, el resultado puede ser distinto.

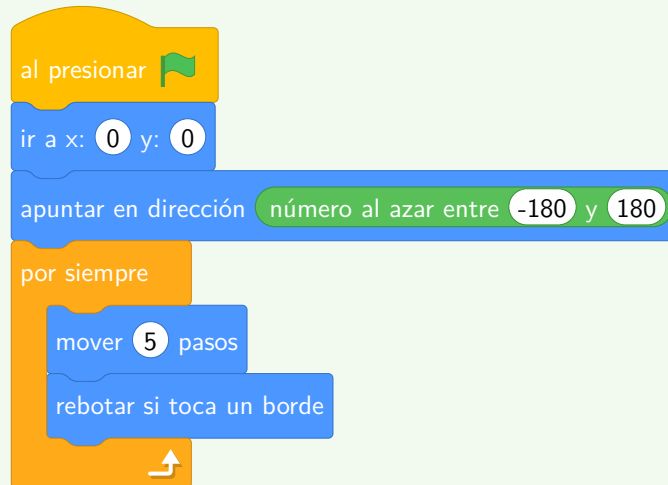
- Para elegir una posición horizontal aleatoria: **fijar x a número al azar entre -220 y 220**
- Para elegir una posición vertical aleatoria: **fijar y a número al azar entre -160 y 160**
- Para hacer aparecer objetos en lugares distintos:
ir a x: número al azar entre -220 y 220 y: número al azar entre -160 y 160

3.4.2. Enemigos y obstáculos

Un enemigo puede moverse sin intervención del jugador. Esto se logra con ciclos, bloques de movimiento y, en muchos casos, rebote en los bordes.

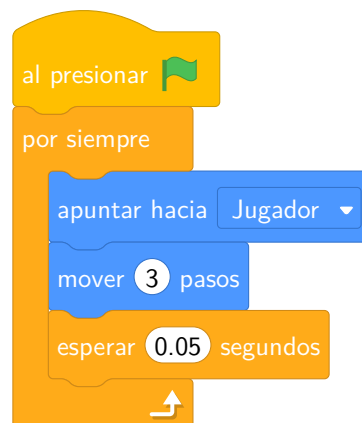
Ejemplo 3.9

El siguiente programa hace que un enemigo se mueva automáticamente.



Persecución del jugador

Para crear un enemigo que siga al jugador, se puede apuntar hacia él y luego avanzar:

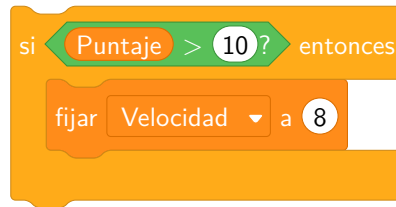


3.4.3. Dificultad progresiva

La dificultad progresiva consiste en hacer que el juego se vuelva más desafiante a medida que el jugador avanza. Algunas formas de hacerlo son:

- aumentar la velocidad de los enemigos;
- reducir el tiempo disponible;
- agregar más obstáculos;

- hacer que los objetos aparezcan con mayor frecuencia;
- cambiar reglas cuando el puntaje aumenta.

**Claim 3.4.1 Velocidad como variable**

Cuando la velocidad de un enemigo cambia durante el juego, conviene guardarla en una variable. Así se puede modificar la dificultad sin reescribir todos los bloques de movimiento.

Ejercicio 3.4 (Actividad: esquivar objetos que caen)

Cree un juego en que objetos caigan desde la parte superior del escenario.

- El jugador se mueve horizontalmente con las flechas.
- Cada obstáculo aparece en una posición x aleatoria.
- El obstáculo cae desde $y = 180$ hasta $y = -180$.
- Si toca al jugador, se resta una vida.
- Si llega abajo sin tocar al jugador, vuelve a aparecer arriba.
- Aumente la velocidad cuando el puntaje sea mayor que 10.

3.5. Comunicación entre personajes

En proyectos con varios personajes, cada uno tiene su propio código. Para coordinar acciones entre ellos se utilizan mensajes. Un mensaje funciona como una señal que un objeto, el escenario o un personaje puede enviar para activar programas en otros elementos.

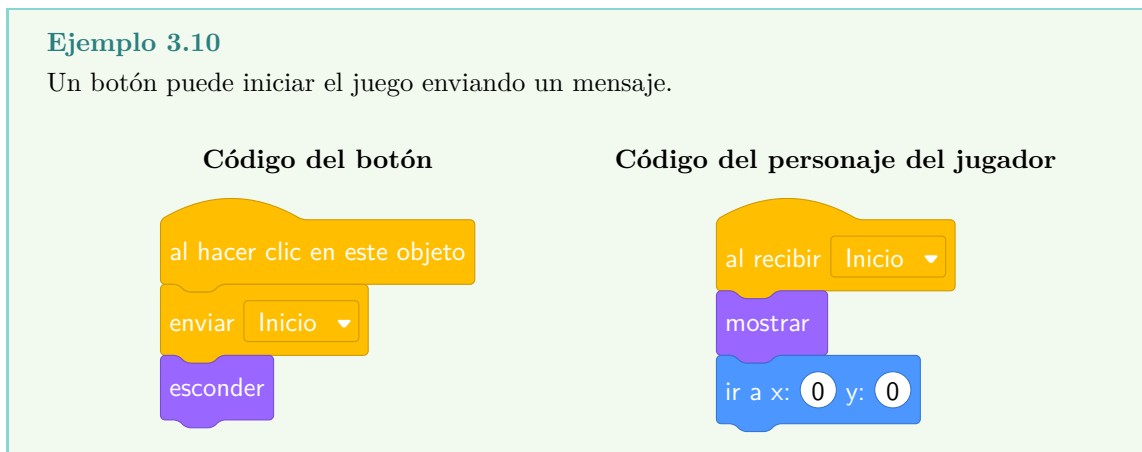
3.5.1. Mensajes

Los bloques principales son:

- **enviar mensaje** envía un mensaje y continúa inmediatamente.
- **al recibir mensaje** ejecuta un programa cuando el personaje o escenario recibe el mensaje.
- **enviar mensaje y esperar** envía un mensaje y espera a que terminen los programas activados por él.

Ejemplo 3.10

Un botón puede iniciar el juego enviando un mensaje.

**3.5.2. Organización del juego mediante mensajes**

Los mensajes ayudan a separar el juego en momentos claros:

- *Inicio*: prepara la partida.
- *Cambio de nivel*: actualiza fondo, enemigos y reglas.
- *Victoria*: muestra animaciones y detiene controles.
- *Derrota*: muestra resultados y detiene enemigos.
- *Reinicio*: vuelve a valores iniciales.

Claim 3.5.1 Nombres de mensajes

Use nombres descriptivos. Es más fácil entender un proyecto con mensajes llamados *Inicio*, *Nivel 2* o *Derrota* que con mensajes como *mensaje1* o *mensaje2*.

3.5.3. Secuencias coordinadas

Los mensajes permiten coordinar cambios de fondo, aparición de enemigos, sonidos y animaciones.

**Ejercicio 3.5** (Actividad: pantalla de inicio con mensaje)

Cree una pantalla de inicio para un juego.

- Agregue un fondo llamado *Portada*.
- Cree un botón llamado *Comenzar*.
- Al presionar la bandera verde, muestre la portada, el título y el botón.
- Al hacer clic en el botón, envíe el mensaje *Inicio*.
- Al recibir *Inicio*, esconda la portada, muestre al jugador e inicialice puntaje, vidas y tiempo.

3.6. Estados y pantallas del videojuego

Un videojuego suele tener varias pantallas: inicio, partida, victoria, derrota y niveles. Cada pantalla corresponde a un estado del juego, es decir, a un momento con reglas y elementos visibles diferentes.

3.6.1. Pantalla de inicio

La pantalla de inicio presenta el juego antes de comenzar. Puede incluir:

- título del juego;
- instrucciones breves;
- botón para comenzar;
- fondo de portada;
- inicialización de variables y posiciones.



3.6.2. Estado de juego

Durante la partida deben estar activos los controles, enemigos, premios, puntaje, vidas y tiempo. Este estado puede iniciarse con un mensaje:



3.6.3. Pantalla de victoria

La victoria ocurre cuando se cumple una condición, por ejemplo alcanzar cierto puntaje o llegar a la meta.



Al recibir el mensaje de victoria, se pueden detener movimientos, cambiar el fondo y mostrar una animación final.

3.6.4. Pantalla de derrota

La derrota puede ocurrir si el jugador pierde todas sus vidas, se acaba el tiempo o toca un enemigo final.



Ejemplo 3.11

El escenario puede encargarse de mostrar la pantalla final.

**3.6.5. Cambio de niveles**

Los niveles pueden representarse con distintos fondos y reglas. Al cambiar de nivel, es posible:

- cambiar el fondo del escenario;
- aumentar la velocidad de los enemigos;
- agregar obstáculos;
- mantener el puntaje;
- reiniciar posiciones de personajes.

**Claim 3.6.1 Estados visibles**

Cada estado del juego debe dejar claro qué personajes se muestran y cuáles se esconden. Esto evita que botones de portada, enemigos o mensajes finales aparezcan durante momentos incorrectos.

Ejercicio 3.6 (Actividad: portada, partida y pantalla final)

Construya un juego con tres estados.

- a) Portada: muestra título, instrucciones y botón de comenzar.
- b) Partida: muestra jugador, enemigos, puntaje, vidas y tiempo.
- c) Pantalla final: muestra victoria o derrota y permite comenzar nuevamente.
- d) Use mensajes para cambiar entre estados.

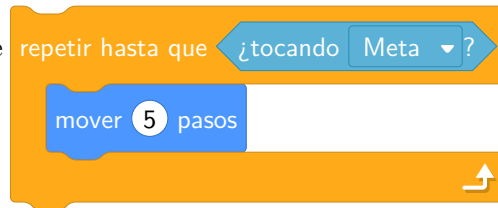
e) Use fondos diferentes para representar cada pantalla.

3.7. Repeticiones avanzadas y control del juego

Los ciclos permiten repetir acciones. En capítulos anteriores se usaron *repetir* y *por siempre*. Ahora se agregan bloques que permiten repetir hasta que ocurra algo, esperar condiciones y detener partes del programa.

3.7.1. El bloque repetir hasta que

El bloque **repetir hasta que**  repite una acción mientras la condición



sea falsa. Cuando la condición se cumple, el ciclo termina.

Ejemplo 3.12

El personaje avanza hasta tocar la meta.



3.7.2. Detención de programas

El bloque **detener**  finaliza todos los programas del proyecto. También existen opciones para detener solo el programa actual u otros programas del mismo objeto.

- *Detener todos*: útil al terminar completamente el juego.
- *Detener este programa*: útil cuando solo se quiere cerrar una secuencia.
- *Detener otros programas en el objeto*: útil cuando un personaje tiene varios ciclos activos.

3.7.3. Esperar condiciones

El bloque **esperar hasta que**  pausa el programa hasta que la condición sea verdadera. Sirve para sincronizar momentos del juego.



3.7.4. Ciclos de juego

Un ciclo de juego organiza la partida en cuatro momentos:

1. **Inicialización:** se fijan variables, posiciones, fondos y disfraces.
2. **Desarrollo:** el jugador controla al personaje y ocurren eventos.
3. **Comprobación:** se revisa si ganó, perdió o cambió de nivel.
4. **Reinicio:** se prepara una nueva partida si el jugador lo solicita.

Claim 3.7.1 Un ciclo principal

En proyectos pequeños, un ciclo *por siempre* puede revisar controles, colisiones y reglas. En proyectos más grandes, conviene dividir el trabajo entre varios personajes y usar mensajes para activar o detener cada parte.

Ejercicio 3.7 (Actividad: persecución con meta)

Cree un juego de persecución.

- a) El jugador debe llegar a una meta.
- b) Un enemigo persigue al jugador.
- c) El juego termina si el jugador toca la meta.
- d) El juego termina si el enemigo toca al jugador.
- e) Use *repetir hasta que*, *esperar hasta que* y algún bloque de detención.

3.8. Clones

Los clones son copias temporales de un personaje que se crean durante la ejecución del programa. Permiten generar muchos enemigos, premios, proyectiles u obstáculos sin tener que duplicar manualmente personajes en el panel de objetos.

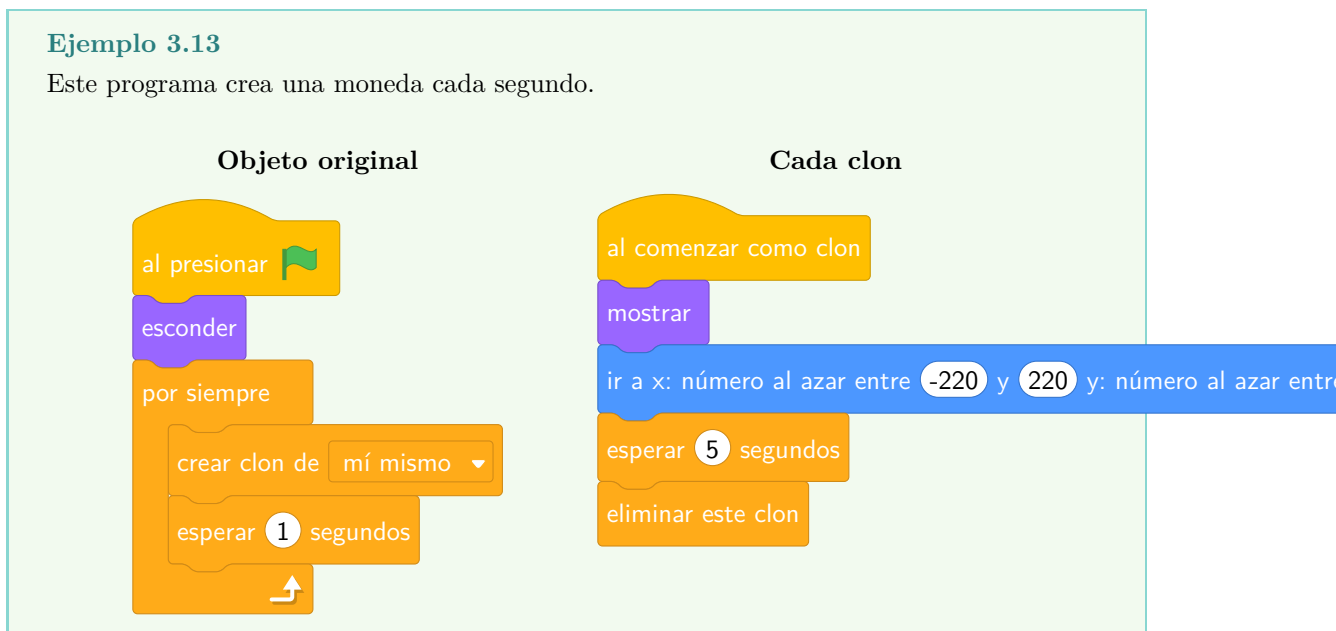
3.8.1. Introducción a los clones

Los bloques principales son:

- **crear clon de** mí mismo ▼ crea una copia del personaje seleccionado.
- **al comenzar como clon** ejecuta el programa que tendrá cada clon al aparecer.
- **eliminar este clon** borra el clon actual.

Ejemplo 3.13

Este programa crea una moneda cada segundo.



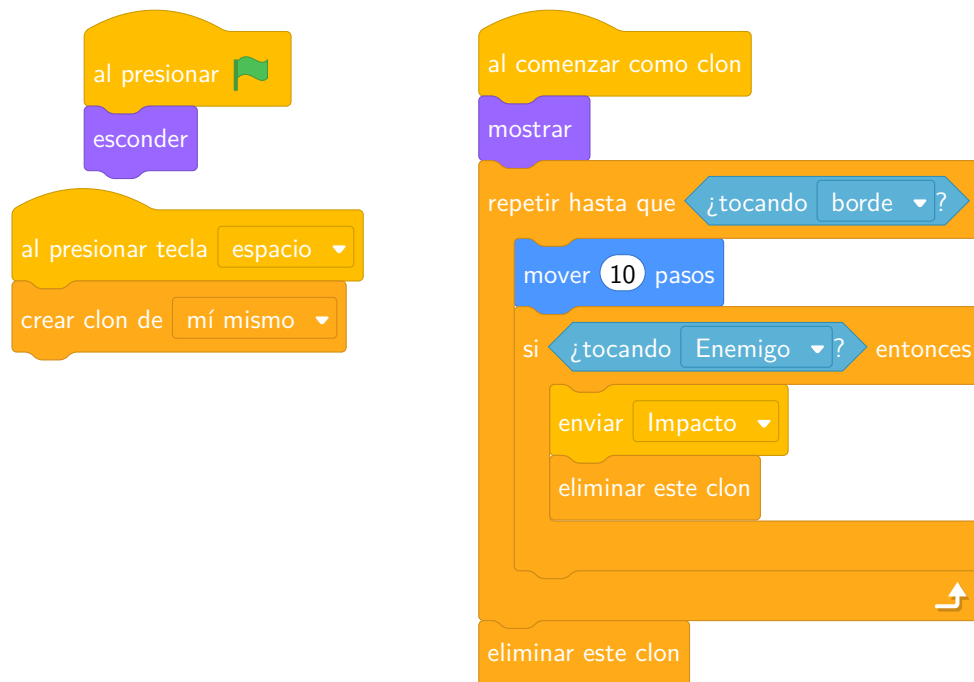
3.8.2. Aplicaciones en videojuegos

Los clones son útiles para:

- generar varios enemigos;
- crear proyectiles;
- hacer aparecer premios;
- producir obstáculos repetidos;
- crear efectos visuales breves, como explosiones o partículas.

Proyectiles

Un proyectil puede crearse al presionar una tecla. Luego, cada clon avanza hasta tocar un borde o un enemigo.



3.8.3. Propiedades de cada clon

Cada clon conserva las propiedades del personaje original en el momento en que fue creado: posición, dirección, tamaño, disfraz y efectos. Después de aparecer, cada clon puede cambiar sus propias propiedades sin modificar directamente a los demás clones.

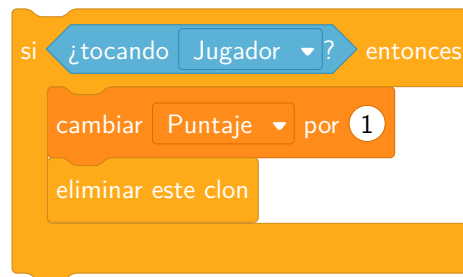
Claim 3.8.1 Preparar antes de clonar

Si se desea que los clones nazcan con posiciones, disfraces o direcciones distintas, conviene ajustar esas propiedades antes de usar `crear clon de mí mismo` o al inicio del bloque

`al comenzar como clon`.

Cantidad de clones

Crear demasiados clones puede volver lento un proyecto. Por eso es importante eliminar los clones cuando ya no cumplen una función.



Ejercicio 3.8 (Actividad: premios con clones)

Cree un juego de recolección usando clones.

- a) El objeto original debe permanecer escondido.
- b) Cada segundo debe crear un clon de premio.
- c) Cada clon debe aparecer en una posición aleatoria.
- d) Si el jugador toca un clon, aumenta el puntaje y el clon se elimina.
- e) Si pasan varios segundos sin recogerlo, el clon desaparece.
- f) Agregue una condición de victoria cuando el puntaje llegue a 10.

El **Propedéutico de la Escuela de Ingeniería Matemática de la Universidad Católica del Maule**, iniciado en 2023, nace con el propósito de potenciar el talento de estudiantes de enseñanza media interesados en la matemática. A través de distintas experiencias de aprendizaje, busca promover el pensamiento crítico, la creatividad y la resolución de problemas, al mismo tiempo que acerca tempranamente a los estudiantes al mundo universitario.

Esta iniciativa forma parte de un compromiso más amplio por proyectar la matemática más allá de la universidad y fortalecer sus vínculos con la comunidad educativa. En este contexto, y en colaboración con la **Escuela de Pedagogía en Matemática**, surge el presente apunte como material de apoyo para el **Taller de Scratch**, dirigido especialmente a profesoras y profesores de enseñanza media interesados en incorporar nuevas herramientas a sus prácticas pedagógicas.

Scratch, desarrollado por el grupo Lifelong Kindergarten del **MIT Media Lab, del Massachusetts Institute of Technology (MIT)**, es un entorno de programación visual que permite crear historias, animaciones, juegos, simulaciones y proyectos interactivos. Su propuesta favorece una aproximación intuitiva a la programación y abre múltiples posibilidades para explorar ideas matemáticas, desarrollar el pensamiento computacional y abordar problemas desde una perspectiva creativa.

Este apunte ha sido concebido como una guía para acompañar a los docentes en la exploración de Scratch y en el diseño de experiencias que puedan ser llevadas al aula y adaptadas a distintos contextos educativos.

*Más que aprender a utilizar una herramienta tecnológica, estas páginas proponen una invitación a **experimentar, crear, programar y descubrir nuevas formas de enseñar y aprender matemática.***